SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A++' Grade
॥वसुधैव कुटुम्बकम्॥

CSE - Academic Year 2026-27
Java Programming Lab | Batch 2025-29

ASSIGNMENT : 27

Name of Student	Anushree Shinde
PRN No.	25070122041
Batch	CSE A - 2
Class	Java Programming Lab
Academic Year & Semester	2025-29 : Semester 3
Date of Submission	

TITLE : Develop a Java application demonstrating socket programming for communication between a client and a server.

Learning Objectives : To establish communication between client and server using Java sockets.

Theory :

Socket programming enables communication over a network using TCP/IP. ServerSocket listens on a port, and Socket connects a client to the server. Input and output streams are used to exchange messages after a connection is established.

Exercise :

1. Create a simple client-server chat application using sockets.

`ChatServer.java :`

```
import java.net.*;
import java.io.*;

public class ChatServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Server is waiting for client...");
        Socket s = ss.accept();
        System.out.println("Client connected.");
        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);
        BufferedReader keyboard = new BufferedReader(
            new InputStreamReader(System.in));

        System.out.println("Client: " + in.readLine());
        System.out.print("Enter reply: ");
        String reply = keyboard.readLine();
        out.println(reply);

        s.close();
        ss.close();
    }
}
```

ChatClient.java :

```
import java.net.*;
import java.io.*;

public class ChatClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5000);
        System.out.println("Connected to server.");
        BufferedReader keyboard = new BufferedReader(
            new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(
            s.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));
```

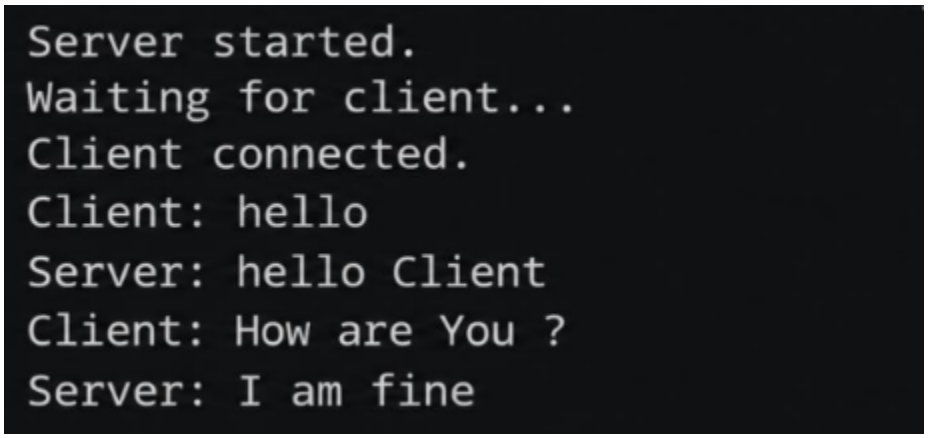
```
        System.out.print("Enter message: ");
        String message = keyboard.readLine();
        out.println(message);

        System.out.println("Server: " + in.readLine());

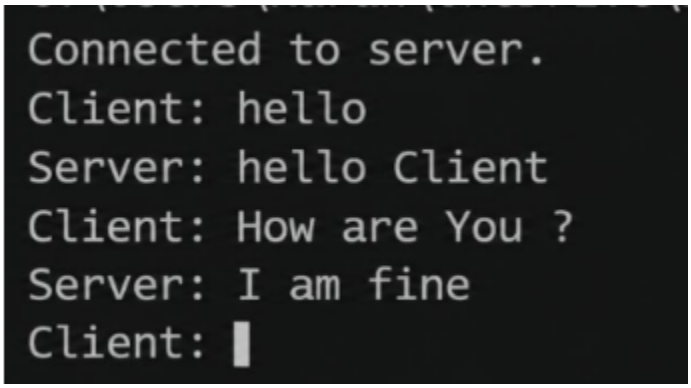
        s.close();
    }
}
```

OUTPUT :

Server side :



Client side :



2. Create an echo system where the server returns the same message sent by the client.

EchoServer.java :

```
import java.io.*;
import java.net.*;
public class EchoServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(6000);
            System.out.println("Echo Server started.");
            System.out.println("Waiting for client...");
            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");
            BufferedReader input = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(
                    socket.getOutputStream(), true);
            String message;
            while ((message = input.readLine()) != null) {
                System.out.println("Client: " + message);
                output.println(message);
                if (message.equalsIgnoreCase("bye")) {
                    break;
                }
            }
            socket.close();
            serverSocket.close();
            System.out.println("Echo server stopped.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

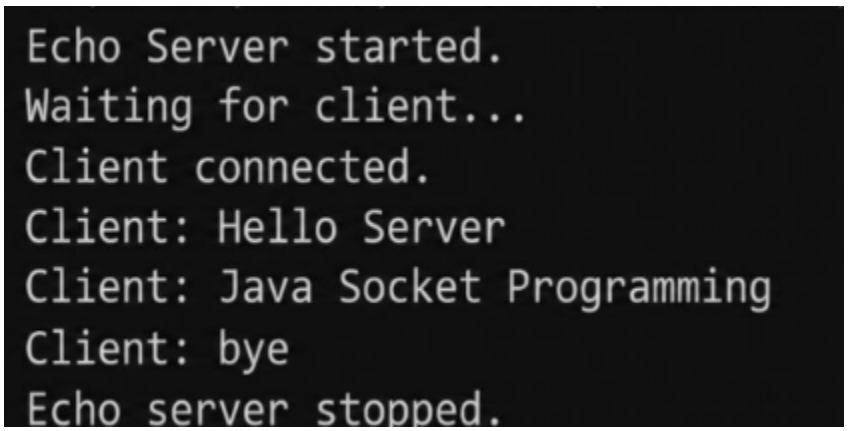
EchoClient.java :

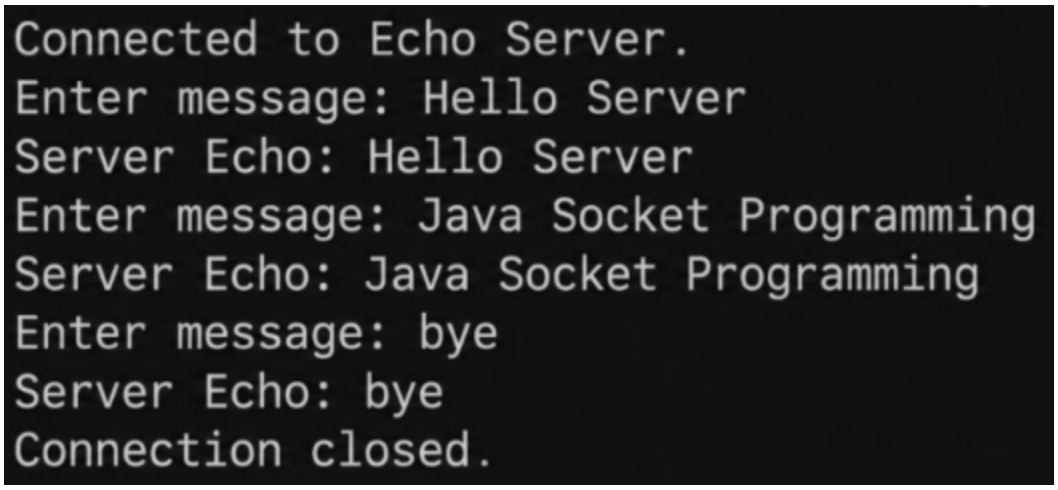
```
import java.io.*;
import java.net.*;

public class EchoClient {
    public static void main(String[] args) {
```

```
        try {
            Socket socket = new Socket("localhost", 6000);
            System.out.println("Connected to Echo Server.");
            BufferedReader keyboard = new BufferedReader(
                    new InputStreamReader(System.in));
            BufferedReader input = new BufferedReader(
                    new InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(
                    socket.getOutputStream(), true);
            while (true) {
                System.out.print("Enter message: ");
                String message = keyboard.readLine();
                output.println(message);
                String response = input.readLine();
                System.out.println("Server Echo: " + response);
                if (message.equalsIgnoreCase("bye")) {
                    break;
                }
            }
            socket.close();
            System.out.println("Connection closed.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

OUTPUT :





Outcome :

- Understood the concept of socket programming in Java.
- Learned to establish client-server communication using Socket and ServerSocket.
- Understood the use of input and output streams for exchanging messages.
- Learned how a server accepts client connections and processes messages.
- Successfully implemented client-server communication and an echo system using Java sockets.

Conclusion :

This assignment demonstrated the use of socket programming for communication between a client and a server. Client-server messaging and an echo application were implemented using Java networking classes and streams. The assignment helped in understanding TCP-based communication, socket connections, and message transmission between networked applications.